



RAIDS

RAID: Redundant Array of Inexpensive Disks

- Combines multiple disks for **faster, larger, and more reliable** storage
- Introduced in late 1980s by UC Berkeley researchers (Patterson, Katz, Gibson)
- Appears as a single disk to the host, **transparent** to software
- Advantages:
 - **Performance:** Parallel I/O speeds
 - **Capacity:** Larger combined storage
 - **Reliability:** Redundancy tolerates disk failures
- Evaluated on **Capacity, Reliability, and Performance**

RAID Interface and Internals

- **RAID to file system:** Appears as a large, linear array of blocks (read/write)
- **Logical vs Physical I/O:**
 - Logical I/O request → RAID maps to one or more physical disk I/Os
 - Example: Mirrored RAID writes 2 physical copies per logical write
- **RAID Hardware:**
 - Separate box connected via SCSI, SATA, etc.
 - Contains microcontroller running firmware
 - Uses volatile memory (DRAM) to buffer data
 - May use non-volatile memory for safe write buffering
 - Specialized logic for parity calculations (in some RAID levels)
- **RAID as a specialized computer:** Processor + memory + disks running RAID software

RAID Fault Model: Fail-Stop

- Disk is either **working** (all blocks readable/writable) or **failed** (completely lost)
- RAID controller can quickly detect when a disk fails
- Doesn't cover silent errors like data corruption or bad sectors
- More complex faults will be discussed later

How to Evaluate a RAID

- Evaluate RAID designs along **3 key axes**:
- **Capacity**
 - Total usable storage after redundancy
 - Example: Mirroring halves capacity $(N \times B) / 2$ ($N \times B$) / 2 $(N \times B) / 2$
- **Reliability**
 - Number of disk failures the RAID can tolerate
 - Based on whole-disk failure model
- **Performance**
 - Depends on workload type
 - Needs workload context for accurate evaluation
 - Next: Overview of RAID Levels 0 (striping), 1 (mirroring), and 4/5 (parity)

RAID Level 0: Striping

- **No redundancy** not truly RAID, but shows max performance & capacity
- **How it works:** Blocks spread (striped) across disks in a round-robin way
- **Stripe:** One block per disk in a row (e.g., blocks 0,1,2,3)
- **Chunk size:** Can be multiple blocks before moving to next disk (e.g., 2 blocks = 8KB chunk size)
- Designed to maximize **parallelism** for large, sequential reads

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Figure 38.1: RAID-0: Simple Striping

Disk 0	Disk 1	Disk 2	Disk 3	
0	2	4	6	chunk size: 2 blocks
1	3	5	7	
8	10	12	14	
9	11	13	15	

Figure 38.2: Striping With A Bigger Chunk Size

Chunk size: Impact on Performance

- **Small Chunk Size**

- Files striped across many disks → high parallelism
- Positioning time = max positioning across all disks (can be slow)

- **Large Chunk Size**

- Less intra-file parallelism → relies on concurrent requests
- Positioning time reduced if file fits on one disk

- **Choosing Chunk Size**

- Depends on workload, hard to optimize
- Common sizes: 4KB (single block) to 64KB
- Example assumes 4KB for simplicity

RAID-0 Analysis

- **Capacity:** Perfect utilization: $N \text{ disks} \times B \text{ blocks} = N \cdot B \text{ blocks of capacity}$
- **Reliability:** Weak point: any single disk failure = data loss
- **Performance:** Excellent: all disks work in parallel to serve I/O requests
- **Two Key Metrics to evaluate RAID**
- **Single-request latency:** How long one I/O request takes
- **Steady-state throughput:** Total bandwidth for many concurrent requests (most important for RAID)

Workload Types for RAID

- **Sequential Workload:**

- Large contiguous data requests (e.g., 1 MB chunk)
- Common in file scanning and backups

- **Random Workload:**

- Small, random data requests (e.g., 4 KB at random addresses)
- Typical in transactional systems like databases

Sequential vs Random Access Performance

- **Sequential Access:**

- Efficient: low seek and rotational delay, more transfer time

- **Random Access:**

- Inefficient: high seek and rotational delay, little transfer time

- **Transfer rates:**

- S = Sequential transfer speed (MB/s)
 - R = Random transfer speed (MB/s)
 - Generally, $S \gg R$

Example Disk Characteristics

- **Given:**
- Average seek time = 7 ms
- Average rotational delay = 3 ms
- Transfer rate = 50 MB/s
- Sequential transfer size = 10 MB
- Random transfer size = 10 KB

Calculating Sequential Transfer Speed (S)

- **Time components for 10 MB transfer:**

- Seek: 7 ms

- Rotation: 3 ms

- Transfer: $10 \text{ MB} / 50 \text{ MB/s} = 200 \text{ ms}$

- **Total time:** $7 + 3 + 200 = 210 \text{ ms}$

- **Sequential throughput:**

- $S = 10 \text{ MB} / 210 \text{ ms} = 47.62 \text{ MB/s}$

- **Interpretation:** Close to peak disk bandwidth because transfer dominates time

Calculating Random Transfer Speed (R)

- **Time components for 10 KB transfer:**

- Seek: 7 ms

- Rotation: 3 ms

- Transfer: $10 \text{ KB} / 50 \text{ MB/s} = 0.195 \text{ ms}$

- **Total time:** $7 + 3 + 0.195 \approx 10.195 \text{ ms}$

- **Random throughput:**

- $R = 10 \text{ KB} / 10.195 \text{ ms} = 0.981 \text{ MB/s}$

- **Interpretation:** Much lower throughput due to overhead of seek and rotation

Summary: Sequential vs Random Throughput

Metric	Sequential (S)	Random (R)
Throughput (MB/s)	47.62	0.981
Ratio S / R	~50×	

- Sequential transfers are ~50 times faster than random transfers
- Highlights importance of workload type on RAID performance

Back To RAID-0 Analysis, Again

- **Latency:**

- Single-block request latency $\approx$ latency of one disk
- RAID-0 redirects requests to individual disks

- **Steady-State Throughput:**

- **Sequential:** Throughput = $N \times S$ (number of disks $\times$ single disk sequential bandwidth)
- **Random:** Throughput = $N \times R$ (number of disks $\times$ single disk random bandwidth)
- **Note:**
 - These represent upper bounds for RAID performance comparisons

RAID Level 1: Mirroring

- **Mirroring:** Store two copies of each block on separate disks
- **Example layout:**
 - Disk 0 & Disk 1 hold identical data
 - Disk 2 & Disk 3 hold identical data
 - Data is striped across these mirrored pairs (RAID-10)
- **Reading:**
 - Can read from either copy for better performance
- **Writing:**
 - Must write to both copies to keep data consistent
 - Writes can happen in parallel on both disks

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

Figure 38.3: Simple RAID-1: Mirroring

RAID-1 Analysis

- **Capacity:** Useful capacity = $(N \times B) / 2$ (mirroring halves capacity)
- **Reliability:**
 - Tolerates at least 1 disk failure
 - Can tolerate up to $N/2$ failures depending on which disks fail (rare)
- **Latency:**
 - **Reads:** Same as single disk (read from one copy)
 - **Writes:** Two physical writes in parallel, slightly slower than single disk write
- **Steady-State Throughput:**
 - **Sequential Write:** $(N/2) \times S$ (half peak bandwidth)
 - **Sequential Read:** Also $(N/2) \times S$ due to disk skipping overhead
 - **Random Read:** $N \times R$ (full bandwidth, reads distributed)
 - **Random Write:** $(N/2) \times R$ (half bandwidth, writes duplicated)
- **Summary:**
 - Mirroring sacrifices capacity but improves reliability
 - Best random read performance among RAID levels

RAID Level 4: Saving Space With Parity

- **RAID-4 Overview:**

- Data striped across multiple disks
- One disk dedicated to parity blocks for redundancy

- **Parity Calculation Using XOR:**

- XOR of all data blocks in a stripe forms the parity block
- Ensures even number of 1s across data + parity bits
- Allows recovery of any single lost block

- **Recovery:**

- Missing data block = XOR of remaining data blocks + parity block

- **Bitwise XOR:**

- Applied bit-by-bit across all blocks to compute parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Figure 38.4: RAID-4 With Parity

RAID 4 Example Structure

- Each parity block (P0, P1, P2, P3) computed from corresponding data blocks
- How Parity is Calculated
 - Parity = XOR of all data blocks in the stripe
 - Ensures even number of 1s in data + parity bits
- Example with bits:
| C0 | C1 | C2 | C3 | Parity (P) |
|----|-----|----|----|-----|
| 0 | 0 | 1 | 1 | 0 (XOR) |
| 0 | 1 | 0 | 0 | 1 (XOR) |
- Parity Invariant
 - The XOR parity bit keeps the number of 1s in each row even, This invariant is critical for parity correctness, helps detect and recover lost data

Block0	Block1	Block2	Block3	Parity
00	10	11	10	11
10	01	00	01	10

RAID-4 Capacity & Reliability

- **Capacity:**

- Uses 1 disk for parity per group of N disks
- Useful capacity = $(N - 1) \times B$ (B = block size)

- **Reliability:**

- Tolerates **1 disk failure** only
- Data lost if more than 1 disk fails

RAID-4 Performance Overview

- **Sequential Read:**

- Uses all data disks except parity disk
- Throughput $\approx (N - 1) \times \text{read speed}$

- **Full-Stripe Write:**

- Writes entire stripe + parity simultaneously
- Throughput $\approx (N - 1) \times \text{write speed}$

- **Random Read:**

- Reads distributed across data disks
- Throughput $\approx (N - 1) \times \text{read speed}$

Random Writes in RAID-4

- **Parity disk is fixed** (dedicated disk for parity): When you write to a single data block, you must also update the parity block on the parity disk.
- **Every random write involves the parity disk.**
- Since all parity updates go to the same disk, this disk becomes a **bottleneck**.
- Result:
 - Writes **serialize on the parity disk**, limiting concurrency.
 - Total performance suffers because parity disk can't handle many writes in parallel.
- Each random write needs **4 I/Os**:
 - Read old data
 - Read parity
 - Write new data
 - Write new parity
- Throughput limited to roughly **half a single disk's speed** (because parity disk is the bottleneck).

RAID-4 Random Writes & Bottleneck

- **Random Writes Challenge:**
- Parity block must be updated to maintain correctness
- Two parity update methods:
 - Additive parity (read all data blocks, XOR with new data)
 - Subtractive parity (update parity using old & new data XOR):
$$P_{\text{new}} = (C_{\text{old}} \oplus C_{\text{new}}) \oplus P_{\text{old}}$$
- **Small-Write Problem:**
 - Parity disk is bottleneck (serializes writes)
 - Each write requires 2 reads + 2 writes (4 I/Os)
 - Random write throughput limited to $\sim R/2$ (half disk speed)
- **Latency:**
 - Read latency $\approx$ single disk latency
 - Write latency $\approx$ twice single disk latency due to multiple I/Os

RAID Level 5: Rotating Parity

- Similar to RAID-4 but **parity blocks are rotated across all disks** instead of being on a single dedicated parity disk.
- This rotation **distributes the parity load evenly** among all disks.
- By **rotating parity blocks**, RAID-5 removes the **single parity disk bottleneck** present in RAID-4.
- This improves write performance by **allowing multiple parity updates to happen in parallel** across different disks.

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

Figure 38.7: RAID-5 With Rotated Parity

RAID-5 Analysis

- **Capacity & Reliability:**

- Same as RAID-4: Useful capacity = $(N - 1) \times B$
- Tolerates 1 disk failure

- **Performance:**

- Sequential reads/writes: Same as RAID-4
- Random reads: Better than RAID-4 (uses all disks in parallel)
- Random writes: Improved over RAID-4 due to **rotated parity enabling parallel parity updates**
- Random write throughput $\approx (N/4) \times R$ MB/s (due to 4 I/Os per write)

- **Market Use:**

- RAID-5 replaced RAID-4 in most systems except where only large writes occur

Summary

	RAID-0	RAID-1	RAID-4	RAID-5
Capacity	$N \cdot B$	$(N \cdot B)/2$	$(N - 1) \cdot B$	$(N - 1) \cdot B$
Reliability	0	1 (for sure) $\frac{N}{2}$ (if lucky)	1	1
Throughput				
Sequential Read	$N \cdot S$	$(N/2) \cdot S^1$	$(N - 1) \cdot S$	$(N - 1) \cdot S$
Sequential Write	$N \cdot S$	$(N/2) \cdot S^1$	$(N - 1) \cdot S$	$(N - 1) \cdot S$
Random Read	$N \cdot R$	$N \cdot R$	$(N - 1) \cdot R$	$N \cdot R$
Random Write	$N \cdot R$	$(N/2) \cdot R$	$\frac{1}{2} \cdot R$	$\frac{N}{4} R$
Latency				
Read	T	T	T	T
Write	T	T	$2T$	$2T$

Figure 38.8: RAID Capacity, Reliability, and Performance



Thank You